Assessment Task 2: Python for Text Processing

Submission details	
Due Date	Time 11:55pm; Date: 22 May 2026 (Friday).
Weighting	Assessment will be marked out of a total of 35 marks. This assessment will contribute to 35% of overall unit grade.
Time to complete	This assessment will take approximately 30 hours to complete
Length & Format	Submit a Jupyter Notebook file. The file must contain all code in code cells, and explanations and answers in text cells. All code cells must be executed, and the output must be visible in the notebook. The file must not be compressed and must be named StudentID.ipynb (replace StudentID with your actual student ID.
Late Penalties	Standard late penalty applies. Unless Special Consideration has been approved, 5% of the total possible mark for the task will be deducted per day for up to 7 days, after which the mark will be 0. The upload deadline is 11:55 PM and a 1-hour grace period applies for technical issues.
How to Submit	Submit in iLearn. Submit the following: • Your completed assignment notebook in `.ipynb` format. • The notebook must include the outputs produced by your final code.
Return of Assessment Grade & Feedback	Grades will be released via iLearn by 5:00 pm on 5 June 2026, with feedback provided within iLearn.
Purpose	In this assignment, you will practise text analysis and duplicate-question classification using the Quora Question Pairs (QQP) dataset. You will begin with corpus-level processing tasks using standard NLP tools, then move to classical machine learning and neural models, ending with Transformer fine-tuning and error analysis. The overall goal is to develop practical skill in implementing, evaluating, and comparing text-processing systems in Python.

Learning Outcomes Assessed	• **ULO1:** Identify real-world applications of artificial intelligence for text and vision. • **ULO2:** Explain key artificial intelligence techniques used in practical systems. • **ULO3:** Design systems using machine learning and deep learning techniques. • **ULO4:** Implement text-processing and classification applications using a programming language.
Skills assessed	Using the methods and tools discussed in lectures and practicals, this assignment assesses your: 1. written and technical communication in notebook form; 2. Python programming for NLP and machine learning; 3. ability to use machine learning and deep learning libraries appropriately; and 4. analysis skills when interpreting results, metrics, and model errors.
How will this task be assessed?	**Individual**. Marks are based on the correctness of your code, outputs, and coding style. A total of 2.5 marks (0.5 per task) are awarded globally across the assignment for both: (1) runnable codes; (2) good coding style: clean, modular code, meaningful variable names, and good comments. If outputs are missing or incorrect, up to 25% of the marks for that task can be deducted. See each task below for the detailed mark breakdown.
Use of AI	Open. You may use generative AI tools within the unit rules. If you do, include a section in your notebook comments beginning with `Use of AI generators in this assignment` and state: • what part of your work was based on generated output; • which tools you used; • which prompts you used; and • what modifications you made to the generated output. This disclosure helps the teaching team assess your work fairly. Reference: Artificial Intelligence Tools and Academic Integrity in FSE.
Short automated extensions	Accepted.

Task Overview

This assignment contains five connected tasks that progress from NLP preprocessing to Siamese neural modelling, and Transformer fine-tuning on QQP. Across all tasks, marks are awarded for both technical correctness and clear, runnable, well-structured notebook code.

About the Dataset

You will use the Quora Question Pairs (QQP) dataset. This dataset contains more than 400,000 question pairs from Quora, labelled according to whether the two questions are semantic duplicates. The dataset is a useful benchmark because it includes informal and noisy text, class imbalance, hard positive examples with low lexical overlap, and hard negative examples with high lexical overlap but different meanings. Working with QQP develops transferable skills in text preprocessing, model comparison, threshold-based reasoning, and error analysis.

```
!pip -q install datasets    # Install the datasets package to access
   the dataset
# add the packages you used, and specify the verion you installed

from datasets import load_dataset
import numpy as np

# 1) Load QQP
ds = load_dataset('glue', 'qqp')

# Use validation set as our test; optionally create a smaller train
   subset for speed
train_ds = ds['train']
eval_ds  = ds['validation']

q1_tr = list(train_ds['question1'])
q2_tr = list(train_ds['question2'])
y_tr  = np.array(train_ds['label'])

q1_te = list(eval_ds['question1'])
q2_te = list(eval_ds['question2'])
y_te  = np.array(eval_ds['label'])
```

Task 1: Top-5 Common Noun (5 marks)

Determine the top-5 most common nouns in the `question1` and `question2` columns respectively.

Write code that returns the top-5 most common nouns found in the questions. Use NLTK's Universal part-of-speech tag set. The expected result format is a descending list of pairs such as `[(noun1, 22), (noun2, 10), ...]`.

Hints

- Concatenate all questions together before processing.
- Use NLTK libraries to tokenize the text and obtain stems if needed as part of your pipeline.
- Apply NLTK's sentence tokenizer before its word tokenizer.
- Use NLTK's part-of-speech tagger with the Universal tagset.
- Use `pos_tag_sents` rather than `pos_tag`.

Marking

2.5 marks are allocated to the correct code and results for `question1`, and 2.5 marks are allocated to the correct code and results for `question2`.

Task 2: Top-5 Common Stem and Non-Stem 2-grams (5 marks)

Determine the top-5 most frequent stemmed and non-stemmed 2-grams for the `question1` and `question2` columns respectively.

Write code that returns the top-5 most frequent bigrams for both stemmed and non-stemmed tokens, along with their normalized frequencies. Frequencies must be normalized by the total number of bigrams and rounded to 4 decimal places. Present results in descending order of frequency, for example `[((‘what’, ‘is’), 0.0105), ((‘what’, ’are’), 0.0053), ...]`.

Hints

- Concatenate all questions together before processing.
- Use NLTK's sentence tokenizer before its word tokenizer.
- Use NLTK libraries to obtain tokens and stems.
- Use the NLTK Porter stemmer to obtain root forms.
- Round normalized frequencies to 4 decimal places.
- When computing bigrams, do not form bigrams across sentence boundaries. For example, if the text is `Sentence 1. And sentence 2.`, then `(‘.’, ‘And’)` is not a valid bigram because the punctuation mark and `And` belong to different sentences.

Marking

2.5 marks are allocated to the correct code and results for `question1`, and 2.5 marks are allocated to the correct code and results for `question2`.

Task 3: Naïve Bayes Classifier (5.5 marks)

The QQP dataset contains pairs of questions with labels indicating whether the two questions are semantic duplicates (1) or not (0).

1. Using a bag-of-words representation, train a Naive Bayes classifier to predict duplicate questions. (2 marks)
2. Report accuracy, precision, and recall on the test set. (1.5 marks)
3. Inspect your confusion matrix. Identify whether false positives or false negatives dominate, and suggest one likely reason based on the dataset. (2 marks)

From this task onward, you are allowed to use a reduced subset of the dataset to limit compute cost. Use the same subset for Tasks 3, 4, and 5 so that model comparisons remain fair. The notebook specifies the following subset sizes:

- `Ntrain = 1000`
- `Ntest = 100`

Use the QQP validation split as the test set, as indicated in the notebook.

Task 4: Siamese Neural Network (7 marks)

In this task you will learn semantic similarity directly from question pairs.

1. Design a Siamese neural network with two identical LSTM encoders that embed each question. (3 marks)
2. Use cosine similarity to classify duplicates and report accuracy and F1-score. (2 marks)
3. Compare your Siamese model with your Naive Bayes model. State which model handles imbalanced errors better in your results and explain why. (2 marks)

Use the same reduced training and test subsets that you selected for Task 3.

Task 5: Transformer-Based Classifier (10 marks)

Instead of handcrafted features or LSTM encoders, you will fine-tune a pre-trained Transformer model, such as BERT or RoBERTa, for QQP.

1. Fine-tune the model for 3 epochs with a learning rate of `2e-5`. (3 marks)
2. Report accuracy, precision, recall, and F1-score. (2 marks)
3. Compare the Transformer results with your Siamese model. State whether the Transformer improves both precision and recall, or mainly one of them, and explain what this suggests about how it captures question meaning. (2 marks)
4. Select one example that your Transformer misclassified and provide a short explanation of why the model might have made that mistake. (3 marks)

Use the same reduced training and test subsets that you selected for Task 3.

Quality Criteria

The following quality criteria apply across the full assignment.

Assignment 2 submission requirements

- your code should run successfully from start to finish in the submitted notebook;
- notebook outputs should correspond to the final version of your code;
- your code should be clear, modular, and appropriately commented;
- variable names and structure should make your approach easy to follow; and
- your analysis and comparisons should directly address the question asked in each task.

Each task specifies a number of marks. The final mark of the assignment is the sum of all marks awarded across the individual tasks and the global code-quality criteria. In addition to the task-specific requirements, the following aspects may be considered when marking your work.

- **Correctness:** provide correct solutions for all questions.
  - Unsatisfactory: only a few questions are answered correctly, or no valid solution is

provided.

  - Pass: correct solutions are provided for about half of the required components.
  - Credit: correct solutions are provided for most of the required components.
  - Distinction: correct solutions are provided for all required components.

- **Code Readability:** write readable, understandable code with a clear coding style.
  - Unsatisfactory: code is difficult to read, variable names are unclear or uninformative, and coding style is poor, for example because comments, whitespace, or indentation are missing or inconsistent.
  - Pass: code is generally readable and the coding style is mostly sound.
  - Credit: code is clearly readable and presented with a clear, consistent coding style.
  - Distinction: coding style is outstanding and the notebook is especially easy to read and follow.
- **Visualization Readability:** where you include figures, charts, confusion matrices, or similar outputs, they should be readable and appropriately presented.
  - Unsatisfactory: visual outputs are poorly presented, for example because they are too small, unclear, or improperly labelled.
  - Pass: visual outputs are generally readable but contain some presentation issues or errors.
  - Credit: visual outputs are clear and use appropriate presentation choices.
  - Distinction: visual outputs are presented exceptionally clearly and are especially easy to interpret.
- **Depth of Analysis:** provide insightful observations, explanations, and analysis of your results.
  - Unsatisfactory: few or no meaningful observations and explanations are provided.
  - Pass: observations are present but remain shallow and only lightly explained.
  - Credit: observations and explanations are generally insightful and relevant.
  - Distinction: observations are interesting and supported by in-depth analysis.

Marking Criteria

Tasks	Marks	Criteria
Global mark	2.5	(0.5 per task) Awarded globally across the assignment for the below: (1) runnable codes; (2) good coding style: clean, modular code, meaningful variable names, and good comments, etc.
Task 1 PoS	5	Excellent (5.0): Correct code and outputs for statistics of both the `question1` and `question2` columns. Poor (0.0): Incorrect or missing implementation.
Task 2 Stem, N-gram	5	Excellent (5.0): Correct code and outputs for statistics of both the `question1` and `question2` columns. Poor (0.0): Incorrect or missing implementation.
Task 3.1 Naive Bayes — Implementation	2	Excellent (2.0): Correctly sets up Bag-of-Words and Naive Bayes, together with corresponding correct outputs. Poor (0.0): Incorrect or missing implementation.
Task 3.2 Naive Bayes — Metrics	1.5	Excellent (1.5): Calculate and report accuracy, precision, and recall correctly. Poor (0.0): Metrics incorrect or missing.
Task 3.3 Naive Bayes — Error Analysis	2	Excellent (2.0): Identifies dominant error type (FP/FN) and explains why it occurs. Poor (0.0): Wrong error type or no explanation.
Task 4.1 Siamese — Network Design	3	Excellent (3.0): Correctly designs Siamese NN with two identical encoders and shared weights, together with corresponding correct outputs. Poor (0.0): Incorrect or incomplete design.
Task 4.2 Siamese — Metrics	2	Excellent (2.0): Calculate and report accuracy and F1-score correctly. Poor (0.0): Metrics incorrect or missing.
Task 4.3 Siamese — Precision/Recall Trade-off	2	Excellent (2.0): Reasonably compares Siamese vs Naive Bayes using results and explains difference in handling errors. Poor (0.0): No comparison or only repeats numbers.

Task 5.1 Transformer — Fine-tuning Setup	3	Excellent (3.0): Correctly describes fine-tuning procedure (epochs, learning rate, pre-trained model), together with corresponding correct outputs. Poor (0.0): Incomplete or incorrect setup.
Task 5.2 Transformer — Metrics	2	Excellent (2.0): Calculate and report accuracy, precision, recall, and F1 correctly. Poor (0.0): Metrics incorrect or missing.
Task 5.3 Transformer — Precision vs Recall Analysis	2	Excellent (2.0): Reasonably identifies whether precision or recall improved more and links it to Transformer properties. Poor (0.0): Restates metrics only or no reasoning.
Task 5.4 Transformer — Misclassified Example	3	Excellent (3.0): Provides real misclassified example and plausible reason. Poor (0.0): No valid example or explanation.